Utica University | Department of Computer Science | Fall 2026
CSC 618: Information Retrieval | Week 10 Assignment

Submission 2: Part B, Controlled Experiment on IR Evaluation

Shaurav Parajuli | November 2026

1. Results

System A is at least as good as System B on every metric and query, so A is the recommended system for all three requirements. All values below are program output (part_b.py, Appendix A). Precision, Recall and F1 are computed on the five returned documents.

Table 1: Per-query scores.

System	Query	Precision	Recall	F1	P@3	AP	RR
A	Q1	0.6000	1.0000	0.7500	0.6667	0.8056	1.0000
A	Q2	0.6000	1.0000	0.7500	0.6667	0.8056	1.0000
A	Q3	0.6000	1.0000	0.7500	0.6667	0.8056	1.0000
B	Q1	0.4000	0.6667	0.5000	0.3333	0.3333	0.5000
B	Q2	0.6000	1.0000	0.7500	0.6667	0.5889	0.5000
B	Q3	0.6000	1.0000	0.7500	0.3333	0.5333	0.5000

Table 2: Aggregate results.

System	MAP	MRR	Mean P	Mean R	Mean F1	Mean P@3	nDCG@5 (Q1)
A	0.8056	1.0000	0.6000	1.0000	0.7500	0.6667	0.9305
B	0.4852	0.5000	0.5333	0.8889	0.6667	0.4444	0.5784

Interpretation. The two systems differ mainly in ordering. On Q2 and Q3 they return the same number of relevant documents, so Precision, Recall and F1 tie, while AP and RR separate them. Only Q1 shows a retrieval gap, where B misses D5. A always ranks a relevant document first; B never does.

Table 3: Which system to use, by search requirement.

Requirement	Evidence	Better
Retrieve more relevant documents	Mean Recall 1.0000 vs 0.8889; difference arises only on Q1	A
Relevant documents near the top	MAP 0.8056 vs 0.4852; MRR 1.0000 vs 0.5000	A
Graded relevance	nDCG@5 for Q1 0.9305 vs 0.5784 (one query only)	A

2. Manual verification

I calculated these by hand and compared them with the program. All match to four decimals.

Table 4: Hand calculations.

Item	Hand calculation	Program
AP, System B, Q3	Relevant at ranks 2, 4, 5: (1/2+2/4+3/5)/3 = 1.6/3	0.5333
MAP, System B	(0.3333+0.5889+0.5333)/3 = 1.4556/3	0.4852

nDCG@5, System B, Q1	Gains 0,3,0,2,0: DCG = $3/\log_2 3+2/\log_2 5$ = 1.8928+0.8614 = 2.7541 (rounded sum); IDCG = $3+2/\log_2 3+1/\log_2 4$ = 4.7619; ratio	0.5784
F1, System B, Q1	P= 2/5, R= 2/3: 2(0.4)(0.6667)/(0.4+0.6667)	0.5000

3. Analysis questions

(a) Higher MAP? System A, 0.8056 vs 0.4852. A consistently places relevant documents earlier, and this gap is much bigger than the gap in mean Recall (1.0000 vs 0.8889), which shows that most of A's advantage comes from ranking rather than from finding more documents.

(b) Higher MRR? System A, 1.0000 vs 0.5000. A's first result is relevant for all three queries. B's first relevant result is always at rank 2, so every B user sees one non-relevant document first.

(c) Do Precision and Recall always agree? No, although they agree here. In this experiment they cannot disagree: every list has 5 documents and every query has 3 relevant ones, so Recall =(5/3)× Precision exactly. They can diverge whenever list length varies. As a hypothetical, a system returning two documents that are both relevant has P= 1.00, R= 0.67, while one returning five with three relevant has P= 0.60, R= 1.00. Which is better depends on the task.

(d) Why can high P@3 go with lower MAP? P@3 ignores everything below rank 3 and the order within the top three; MAP considers every relevant document. This pattern does not appear in my results, since A is never below B, but Q2 is close: both have P@3 = 0.6667, yet AP is 0.8056 for A and 0.5889 for B. A hypothetical with six relevant documents shows the full effect: X finds three at ranks 1–3 and no others (P@3 = 1.000, AP = 0.500), while Y finds all six at ranks 1, 3, 4, 6, 7, 8 (P@3 = 0.667, AP = 0.758).

(e) Higher nDCG@5 for Q1? System A, 0.9305 vs 0.5784. A places the grade-3 D1 first, while B discounts it by placing it second and omits grade-1 D5. Grades do not change the winner, but they change the size of the gap. With binary gains the scores would be 0.9060 and 0.4982, a gap of 0.4078; with grades the gap is 0.3521. It narrows because B does retrieve the two most valuable documents and misses only the least valuable one, which graded relevance penalises lightly.

(f) Why the same queries and judgments? So that the system is the only thing that varies. Different queries could be easier or harder, and different judgments would change what counts as correct. Score differences then could not be attributed to the systems.

(g) Limitations of three queries and eight documents? A single query can change the mean a lot. No statistical test can establish much: with three queries, even with A ahead on all three, the smallest possible two-sided sign-flip p-value is 2/8 = 0.25. Each system returns 5 of 8 documents, so even random lists would often appear to do well, and there are too few documents to show behaviour deep in a ranking.

(h) Why not evidence about a real university search engine? The rankings are hypothetical, not output from a running engine. The judgments are for teaching, not from a user study. The documents and queries are about IR and machine learning, not courses, admissions or campus services. Real engines also face scale, freshness, duplicates, misspellings and diverse user intent. Nothing was measured on real users, or on speed or satisfaction.

Appendix A: Code

ir_metrics.py

```
"""IR evaluation metrics shared by Part B and Part C.

A ranked list is a list of document IDs, best first. 'rel' is the set of relevant IDs.
""" import
math


def precision(ranked, rel): return sum(d in rel for d in ranked) / len(ranked) if ranked else 0.0


def recall(ranked, rel): return sum(d in rel for d in ranked) / len(rel) if rel else 0.0


def f1(ranked, rel):
    p, r = precision(ranked, rel), recall(ranked, rel) return 2 * p * r / (p + r) if p + r else
    0.0


def precision_at_k(ranked, rel, k):
    """The denominator stays k even if fewer than k documents were returned.""" return sum(d in rel for d in ranked[:k]) /
    k


def reciprocal_rank(ranked, rel):
    for rank, doc in enumerate(ranked, 1):
        if doc in rel:
            return 1 / rank return 0.0


def average_precision(ranked, rel):
    """Relevant documents that were not retrieved contribute 0 (division by |rel|).""" hits, total = 0, 0.0 for rank, doc in
    enumerate(ranked, 1):
        if doc in rel:
            hits += 1 total += hits / rank return total /
    len(rel) if rel else 0.0


def dcg(gains):
    return sum(gain / math.log2(rank + 1) for rank, gain in enumerate(gains, 1))


def ndcg_at_k(ranked, grades, k):
    """ 'grades' maps doc -> gain; unjudged documents have gain 0.

    The ideal list is built from all judged documents, not just the retrieved ones.
    """ actual = dcg([grades.get(d, 0) for d in ranked[:k]]) ideal =
    dcg(sorted(grades.values(), reverse=True)[:k]) return actual / ideal if ideal else
    0.0


def mean(values):
    values = list(values) return sum(values) / len(values) if values else 0.0
```

part_b.py

```
"""Part B: controlled comparison of System A and System B.

Usage: python part_b.py <output_dir>

""" import csv
import json
import sys
```

```
from ir_metrics import (
    average_precision, dcg, f1,
    mean, ndcg_at_k, precision,
    precision_at_k, recall,
    reciprocal_rank,
)

DOCUMENTS = {
    "D1": "Neural networks and deep learning methods for image classification",
    "D2": "BM25 ranking functions for information retrieval",
    "D3": "Convolutional neural networks for image recognition",
    "D4": "Language models and probabilistic document retrieval",
    "D5": "Image classification using machine learning",
    "D6": "BM25, query likelihood, and document ranking",
    "D7": "Unigram language models for text search",
    "D8": "Web crawling and link analysis",
}
QUERIES = {
    "Q1": "Neural networks for image classification",
    "Q2": "BM25 information retrieval",
    "Q3": "Language models and document retrieval",
}
RELEVANT = {"Q1": {"D1", "D3", "D5"}, "Q2": {"D2", "D4", "D6"}, "Q3": {"D4", "D6", "D7"}}
GRADES_Q1 = {"D1": 3, "D3": 2, "D5": 1} RUNS = {
    "A": {
            "Q1": ["D1", "D2", "D3", "D5", "D4"],
            "Q2": ["D2", "D1", "D4", "D6", "D3"],
            "Q3": ["D4", "D2", "D6", "D7", "D1"],
    },
    "B": {
            "Q1": ["D2", "D1", "D4", "D3", "D6"],
            "Q2": ["D1", "D4", "D2", "D5", "D6"],
            "Q3": ["D1", "D4", "D3", "D7", "D6"],
    },
}
METRICS = ["Precision", "Recall", "F1", "P3", "AP", "RR"]


def per_query_rows():
    rows = [] for system, run in RUNS.items(): for query,
    ranked in run.items():
                rel = RELEVANT[query] rows.append(
                    {
                        "System": system,
                        "Query": query,
                        "Precision": precision(ranked, rel),
                        "Recall": recall(ranked, rel),
                        "F1": f1(ranked, rel),
                        "P3": precision_at_k(ranked, rel, 3),
                        "AP": average_precision(ranked, rel),
                        "RR": reciprocal_rank(ranked, rel),
                    }
                ) return
    rows
```

```
def aggregate(rows):
    results = {} for system, run in RUNS.items():
        selected = [row for row in rows if row["System"] == system] gains = [GRADES_Q1.get(doc, 0)
        for doc in run["Q1"][:5]] results[system] = {
            "MAP": mean(row["AP"] for row in selected),
            "MRR": mean(row["RR"] for row in selected),
            "mean_P": mean(row["Precision"] for row in selected),
            "mean_R": mean(row["Recall"] for row in selected),
            "mean_F1": mean(row["F1"] for row in selected),
            "mean_P3": mean(row["P3"] for row in selected),
            "gains_Q1": gains,
            "DCG5_Q1": dcg(gains),
            "IDCG5_Q1": dcg(sorted(GRADES_Q1.values(), reverse=True)),
            "nDCG5_Q1": ndcg_at_k(run["Q1"], GRADES_Q1, 5),
        } return
    results


def main(out):
    rows = per_query_rows() results =
    aggregate(rows)

    with open(f"{out}/partb_table.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, rows[0].keys()) writer.writeheader()
        writer.writerows(rows)
    json.dump({"rows": rows, "agg": results}, open(f"{out}/partb_results.json", "w"), indent=1)

    print(f"{'System':6} {'Query':5} " + " ".join(f"{m:>9}" for m in METRICS)) for row in rows:
        values = " ".join(f"{row[m]:9.4f}" for m in METRICS) print(f"{row['System']:6} {row['Query']:5} {values}")
    for system, values in results.items():
        print( f"System {system}: MAP={values['MAP']:.4f} MRR={values['MRR']:.4f} "
            f"nDCG@5(Q1)={values['nDCG5_Q1']:.4f}"
        )


if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else ".")
```